GHRAB VOC · ENGINEERING

Production Architecture & Migration Plan

From single-operator lab tool to a persistent, shared, authenticated multi-team platform — full decision record with alternatives, trade-offs, use cases, and confirmation gates for every choice.

Document — VOC-ARCH-001, v1.0

Date — 17 July 2026

Scope — backend/ · frontend/ · deployment topology · identity · data platform

Status — Proposed, pending review

§0 Contents

§1 Executive summary

Keep the stack, evict the state.

The VOC platform's framework choices — FastAPI, React + Vite, litellm, the seven-agent pipeline — are sound and survive this migration untouched. What does not survive is everything that assumes a single process and a single anonymous operator: module-global session state, an in-memory rate limiter, pipeline runs on request threads, SQLite, raw provider keys in `.env`, and the absence of any identity layer.

The target is a three-tier stateless application (edge, API replicas, job workers) around three stateful services (Postgres for durable truth, Redis for ephemeral coordination, object storage for files), with a LiteLLM Proxy as the only network path to LLM providers, and OIDC against the corporate identity provider with team-scoped role-based access control. Analysis runs become durable, observable jobs whose progress every authorized teammate can watch live — including users who join mid-run or refresh their browser.

Delivery is five phases, each independently shippable, each with an explicit confirmation gate. Identity lands before the deepest change (worker extraction) so it lands on solid ground. The known test gap — one backend test file, zero frontend tests — is treated as a hard prerequisite, not a footnote.

§2 Current-state assessment: the four ceilings

Every decision in §4 exists to retire one of these four ceilings. Nothing is added that does not.

CEILING 1 — PROCESS-LOCAL STATE

`backend/api/state.py` holds the current analysis result, run deduplication, and the active dataset as module globals behind a `threading.Lock`; `backend/core/ratelimit.py` is a dict of sliding windows in one process. **Consequence:** with two API replicas — or merely `uvicorn --workers 2` — users see different "current" runs, duplicate-run protection stops working, and rate limits are counted per-process, i.e. wrong.

CEILING 2 — PIPELINE RUNS INSIDE THE REQUEST PROCESS

`backend/api/streams.py` executes the seven-agent pipeline on a `threading.Thread` and bridges progress into SSE via a stdlib `queue.Queue`. **Consequence:** any deploy, crash, or restart kills every in-flight analysis with no record; one long run competes with all interactive traffic for the only process everyone shares; the sync/async bridge is a recurring source of stream-hang bugs.

CEILING 3 — NO IDENTITY

The only signal a request carries is its IP address (acknowledged in `ratelimit.py`'s own docstring). **Consequence:** no login, no roles, no per-team data separation, no answer to "who launched this run" or "who asked the chat agent about our weakest systems". Unshippable to multiple teams.

CEILING 4 — SQLITE, RAW KEYS, NO MIGRATIONS

One single-writer file database; seven LLM provider keys loaded from `.env` into the app process; schema changes applied by hand. **Consequence:** write contention under concurrent teams, key rotation requires an app redeploy, and no repeatable path from schema N to N+1.

These are design debts consciously taken by a lab tool — the code's own comments say so. The point of this document is that they are the *complete* list of what must change: the migration is precise, not a rewrite.

§3 Target topology walkthrough

Read the diagram top to bottom as a request's journey; every arrow crossing a band boundary is a network hop with authentication on it.

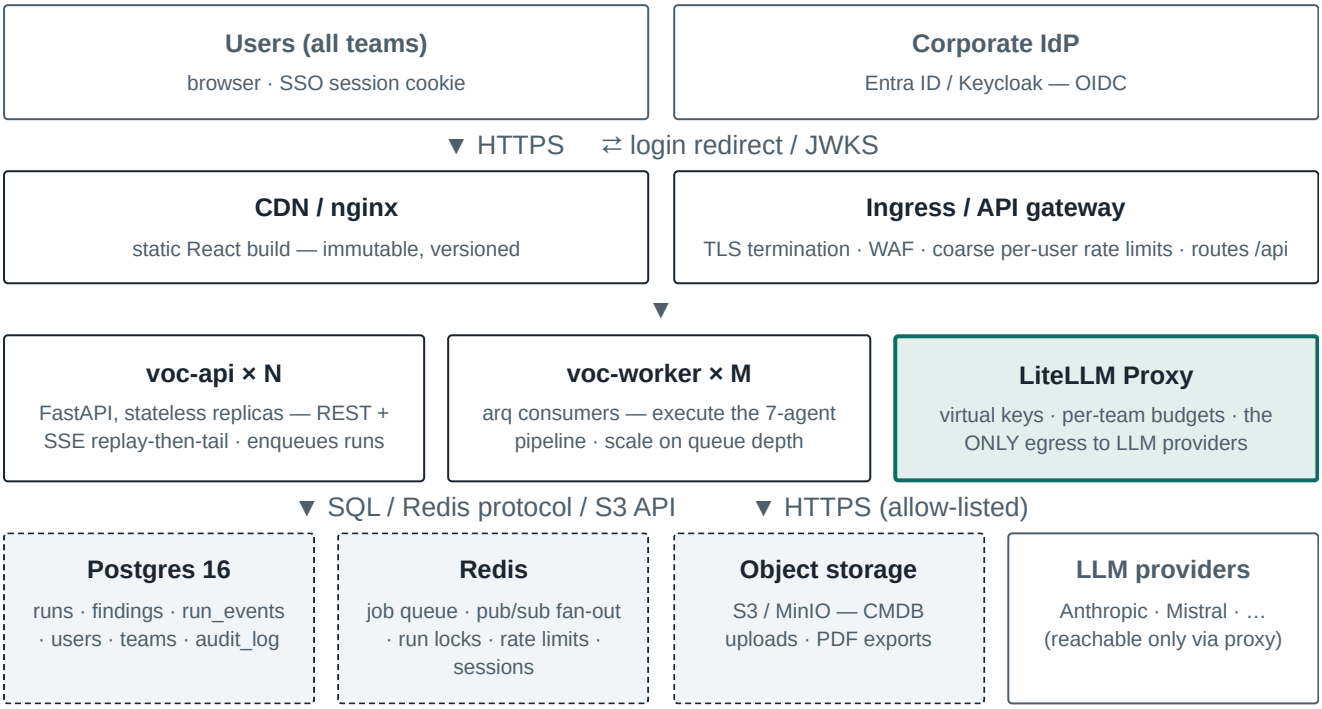

FIG. 1 — TARGET PRODUCTION TOPOLOGY. STATELESS TIERS SCALE HORIZONTALLY; STATE LIVES ONLY IN THE BOTTOM BAND.

Walkthrough, component by component

Edge. The React build is already static Vite output; it moves to a CDN or nginx and out of the API's concern entirely. The ingress terminates TLS, applies coarse per-user rate limits, and routes `/api` to the API pool. Frontend and API stay same-origin, so no CORS surface opens.

API tier. FastAPI, unchanged as a framework, made stateless: every module-global in `api/state.py` moves to Postgres (durable facts) or Redis (ephemeral coordination). Replicas become interchangeable; scaling is a replica count.

Worker tier. The pipeline leaves the request process. `POST /analyze` becomes: acquire the per-dataset Redis lock, insert a `runs` row as `queued`, enqueue, return `202` with the run id. Workers execute the orchestrator exactly as today but publish each progress event to Redis pub/sub *and* append it to `run_events`. Workers survive API deploys and scale independently.

LLM gateway. The application already speaks litellm, so pointing it at a LiteLLM Proxy is a configuration change with outsized returns: provider keys leave the application, teams get budgets and spend dashboards, and the network can enforce that nothing else egresses to the internet.

Stateful band. Postgres holds truth, Redis holds "now", object storage holds files. Nothing above this band retains state between requests — that single property is what makes every other box replaceable, scalable, and deploy-safe.

§4 Decision records

Each record: the decision, why, the alternatives actually weighed with pros and cons, when the choice pays off (use cases), and a concrete confirmation that it is working.

D1 Application framework — keep FastAPI

FastAPI stays. The framework was never the ceiling; process-local state was.

REASONS

- Native async is load-bearing here: SSE streaming, concurrent LLM calls, chat streaming all sit on it.
- Pydantic models are already the API contract and the agents' structured-output layer — shared for free.
- The team knows it; the code is idiomatic; a rewrite adds months and zero retired ceilings.

ALTERNATIVES CONSIDERED

OPTION	PROS	CONS	VERDICT
FastAPI (keep)	Async-native, Pydantic contracts, in place, team fluency	Fewer batteries than Django (no admin, no ORM-integrated auth)	**CHOSEN**
Django + DRF	Admin site, mature auth, migrations built in	Sync-first heritage; SSE/streaming and async LLM fan-out fight the framework; full rewrite of routes and serializers	Rejected — pays rewrite cost to lose the async fit
NestJS / Node	One language across stack	Abandons pandas/networkx — the analytical core — or splits it into a second service anyway	Rejected — the Python data stack is the product
Go / Spring	Raw performance, strong typing	Performance is not the bottleneck (LLM latency is); total rewrite; no data-science ecosystem	Rejected — optimizes the wrong axis

USE CASES SERVED

Streaming analysis progress to many viewers; concurrent multi-provider LLM calls; typed request/response contracts consumed by the TypeScript frontend.

> **CONFIRM** — after Phase 2, run two replicas behind nginx locally; both must serve identical `/api/runs` state and a run started via replica A must stream on replica B.

D2 Primary database — Postgres 16 replaces SQLite

Managed Postgres, single primary + one replica. SQLAlchemy layer ports via connection string; Alembic formalizes the schema.

REASONS

- Concurrent writers: SQLite serializes writes process-wide — fatal once N replicas and M workers write runs, events, and audit rows simultaneously.
- Row and advisory locks give correct semantics for run bookkeeping under concurrency.
- Mature operational story: WAL archiving, point-in-time recovery, replicas, every cloud offers it managed.
- The code is already prepared: `DATABASE_URL` is env-driven and `db/session.py` branches on the scheme.

ALTERNATIVES CONSIDERED

OPTION	PROS	CONS	VERDICT
Keep SQLite	Zero ops, already works	Single writer, file on one node contradicts stateless replicas, no managed backup story	Rejected — is Ceiling 4
Postgres	Concurrency, locks, PITR, managed everywhere, SQLAlchemy-compatible	An operational dependency to run and patch	**CHOSEN**
MongoDB	Flexible documents for agent output	Data is relational (assets ↔ findings ↔ runs ↔ teams); loses FK integrity and the existing SQLAlchemy layer	Rejected — schema is the asset

USE CASES SERVED

Many teams writing concurrently; durable run history and audit trail; per-team row scoping; compliance-grade backup/restore.

> **CONFIRM** — migration script moves `voc.db` to Postgres with row-count parity per table; restore drill from a PITR backup succeeds in staging before Phase 2 closes.

D3 Coordination layer — Redis for queue, pub/sub, locks, rate limits, sessions

One Redis instance carries all ephemeral shared state. Nothing durable ever lives only in Redis.

REASONS

- Five needs (job queue, live event fan-out, run locks with TTL, rate-limit windows, session store) are all ephemeral-coordination shaped — one well-understood service covers all five.
- TTL-based locks are the correct distributed replacement for `check_duplicate()`: a crashed worker's lock expires instead of wedging a dataset forever.
- Losing Redis loses only "now" — queued jobs are re-enqueueable from Postgres `runs` rows, sessions force a re-login, live tails reconnect. Acceptable blast radius by construction.

ALTERNATIVES CONSIDERED

OPTION	PROS	CONS	VERDICT
Redis	Covers all five needs, ubiquitous, managed offerings, tiny ops surface	Single instance is a soft SPOF (mitigated: nothing durable lives there)	**CHOSEN**
RabbitMQ / Kafka	Stronger delivery guarantees, replayable log (Kafka)	Covers one of five needs; Kafka's retention duplicates what `run_events` in Postgres already provides; heavy ops	Rejected — wrong size
Postgres-as-queue (SKIP LOCKED)	One less service	No pub/sub fan-out for SSE, polling latency, mixes OLTP and queue load	Rejected — fan-out is the hard requirement

USE CASES SERVED

All-replica live streaming of one run; exactly-one-run-per-dataset; shared rate limiting; instant session revocation.

> **CONFIRM** — kill Redis mid-run in staging: the run fails visibly (not silently), the UI shows a clear error, re-launch succeeds after restart, and no Postgres data is lost.

D4 Job execution — arq workers replace in-request threads

Pipeline runs execute in dedicated arq worker processes consuming from Redis; same container image as the API, different entrypoint.

REASONS

- Retires Ceiling 2: deploys stop killing runs; a long analysis no longer pins the interactive process.
- arq is asyncio-native — the orchestrator's async LLM calls run unmodified; Celery would wrap them in sync workers or require its newer async layer for no gain.
- Redis is already present (D3); arq adds no new infrastructure.
- Workers scale on queue depth (KEDA), independently of interactive traffic.

ALTERNATIVES CONSIDERED

OPTION	PROS	CONS	VERDICT
arq	Asyncio-native, Redis-native, tiny, retries/timeouts built in	Smaller community than Celery; no beat scheduler (not needed)	**CHOSEN**
Celery	Battle-tested, rich ecosystem, periodic tasks	Sync-first worker model vs an async codebase; broker abstraction, beat, and routing machinery bought and unused	Rejected — pays for machinery this app never uses
FastAPI BackgroundTasks	Zero new pieces	Still in-process: dies with the replica, invisible to other replicas — re-creates Ceiling 2 exactly	Rejected — not a job system
K8s Jobs per run	Strong isolation	Pod cold-start per analysis, YAML lifecycle management, harder local dev	Rejected — latency and complexity

USE CASES SERVED

Deploy during a live analysis; five teams launching runs simultaneously; retrying a run that died on a provider outage; cancel-run semantics with a clean terminal state.

> **CONFIRM** — start a run, deploy a new API image mid-run: the run completes and every connected browser's stream survives. Kill a worker mid-run: the run is marked `failed` with its partial event history intact, and the dataset lock releases within its TTL.

D5 Live updates — SSE with durable replay, not WebSockets

Keep Server-Sent Events; make them durable. Events get sequence numbers, persist to `run_events`, fan out via Redis pub/sub; clients replay-then-tail with `Last-Event-ID`.

REASONS

- Traffic is strictly server→client (progress, chat tokens); SSE matches the shape and the frontend already speaks it.
- Auto-reconnect with `Last-Event-ID` is built into the browser `EventSource` — resume comes nearly free once events are numbered and stored.
- Plain HTTP: works through corporate proxies and the ingress with no upgrade-path special-casing.
- The durable half (Postgres `run_events`) is what gives late joiners full history — the transport alone was never the gap.

ALTERNATIVES CONSIDERED

OPTION	PROS	CONS	VERDICT
SSE + pub/sub + replay	Matches one-way traffic, native reconnect, already in codebase, proxy-friendly	One-way only (fine: commands are POSTs)	CHOSEN
WebSockets	Bidirectional, single connection for everything	Buys bidirectionality nobody needs; hand-rolled reconnect/replay; stateful connections complicate LB draining	Rejected — more machinery, same outcome
Polling	Trivially simple	Latency vs load trade-off at multi-team scale; chat token streaming infeasible	Rejected — wrong UX for live runs

USE CASES SERVED

Teammate opens a run 4 minutes in and sees the full timeline then live tail; browser refresh loses nothing; laptop sleeps and resumes mid-run.

> **CONFIRM** — open a run stream, force-kill the connection, reconnect with the last seen event id: zero gaps, zero duplicates in the event sequence. Late joiner's rendered timeline is byte-identical to an always-connected client's.

D6 Authentication — OIDC via corporate IdP, BFF session pattern

The backend is the confidential OIDC client (authorization-code + PKCE). Browser holds only an `httpOnly; Secure; SameSite=strict` session cookie; sessions live in Redis. No tokens in JavaScript.

REASONS

- Delegating authentication to the IdP the organization already runs means MFA, lockout, offboarding, and password policy are inherited, not rebuilt — and offboarding an employee revokes VOC access automatically.
- BFF over tokens-in-SPA: XSS in a vulnerability-analysis tool must not be able to exfiltrate a bearer token. httpOnly cookies remove that class.
- SSE and chat streams authenticate with the same cookie for free — no token-refresh choreography inside `EventSource`, which does not support custom headers.
- Server-side sessions in Redis give instant revocation — kill a session, it is dead on the next request.

ALTERNATIVES CONSIDERED

OPTION	PROS	CONS	VERDICT
OIDC + BFF sessions	Inherited MFA/offboarding, no tokens in JS, instant revocation, SSE-compatible	Backend owns session plumbing; needs Redis (already present)	**CHOSEN**
JWT in SPA (implicit/code in browser)	Stateless backend, common pattern	Tokens readable by any XSS; EventSource cannot send Authorization headers; revocation requires denylists that re-invent sessions	Rejected — wrong risk profile for this tool
Homegrown username/password	No IdP dependency	Owns password storage, MFA, reset flows, lockout — permanent liability; duplicate identity when the org already has one	Rejected — never build auth an IdP already provides

USE CASES SERVED

Single sign-on with existing corporate accounts; automatic access removal on offboarding; session kill during an incident; auditable identity on every action.

> **CONFIRM** — login round-trip via the IdP works behind the ingress; deleting the Redis session 401s the very next request; `document.cookie` in the console shows no session material; an SSE stream opened without a session is refused.

D7 Authorization — teams as tenancy, three roles, one enforcement point

Every dataset, run, finding, and event carries `team_id`. A single scoped-session dependency injects the team filter into every repository query. Roles — viewer, analyst, admin — map from IdP group claims.

REASONS

- Row-level scoping through one dependency means tenancy is enforced in one auditable place, not re-remembered per endpoint — the class of "forgot the filter on one route" bugs is designed out.
- Three roles cover the real personas (management views, analysts operate, admins govern) without inventing a permission matrix nobody will maintain.
- Roles from IdP groups mean team membership is managed where HR/IT already manage it.
- Cross-team visibility is an explicit grant recorded in the DB — default isolation, deliberate sharing.

ALTERNATIVES CONSIDERED

OPTION	PROS	CONS	VERDICT
Shared DB, team_id row scoping	One schema, one migration path, simple ops, easy cross-team grants	Scoping bug = data leak (mitigated: single enforcement point + tests)	**CHOSEN**
Database-per-team	Hard isolation	N× migrations, N× backups, cross-team reporting becomes ETL; operational cost grows with every team	Rejected — ops cost scales with adoption
Full RBAC/ABAC engine (OPA, Casbin)	Arbitrarily expressive policy	Three roles and one tenancy dimension do not need a policy language; adds a system to learn and operate	Rejected — expressiveness without a customer

USE CASES SERVED

SOC and infrastructure teams each see only their datasets; a director gets viewer access across explicitly granted teams; an admin answers "who exported last month's report".

> **CONFIRM** — automated test: user in team A requests every list/detail endpoint for team B resources and receives 404/403 on all of them; the audit log records the attempts.

D8 LLM access — LiteLLM Proxy as the single gateway

All LLM traffic (workers' pipeline calls, API's chat streaming) goes through a LiteLLM Proxy. Provider keys exist only in the proxy's secret mount; the application holds one virtual key per environment.

REASONS

- The application already speaks litellm — pointing `api_base` at the proxy is the cheapest high-leverage change in the whole design.
- Per-team virtual keys give budgets, spend dashboards, and rate limits per team — LLM cost, today's biggest unpriced risk, becomes governed.
- Key rotation becomes a proxy-side operation with zero application deploys.
- Enables the network posture in §6: only the proxy egresses to the internet — provable non-exfiltration for a tool that ingests the organization's CMDB.
- Central request logging: every prompt/response crosses one audited chokepoint.

ALTERNATIVES CONSIDERED

OPTION	PROS	CONS	VERDICT
Direct provider calls (status quo)	One less service	Seven keys in every app process, no budgets, no central log, every pod needs internet egress	Rejected — is Ceiling 4's key problem
LiteLLM Proxy	Drop-in for existing litellm client, budgets/virtual keys/fallbacks/logging built in	One more service to run and patch	**CHOSEN**
Custom gateway service	Exact fit	Months to reach feature parity with an off-the-shelf proxy; permanent maintenance	Rejected — undifferentiated heavy lifting

USE CASES SERVED

Per-team monthly token budgets with graceful degradation at the cap; provider fallback when one is down; finance asking "what did team X spend on LLMs in Q3"; security asking "show me every prompt that left the building".

> **CONFIRM** — grep the deployed app environment for provider keys: none present. Exhaust a test team's budget: chat and analysis return the clear budget-exceeded message, other teams unaffected.

D9 File storage — object storage replaces the container filesystem

CMDB CSV uploads and generated PDF reports move to S3 (or MinIO on-prem), keys namespaced by team; large transfers use presigned URLs.

REASONS

- Container filesystems are ephemeral by design; uploads must outlive pods, deploys, and node failures.
- Presigned URLs keep multi-megabyte CSV uploads and PDF downloads off the API's request path.
- Bucket versioning + lifecycle rules give retention policy for compliance artifacts with no app code.

ALTERNATIVES CONSIDERED

OPTION	PROS	CONS	VERDICT
Persistent volume	Familiar file paths	Ties pods to nodes/zones, complicates multi-replica reads, no presigned access, weak retention tooling	Rejected
S3 / MinIO	Durable, replica-agnostic, presigned URLs, versioning, on-prem option exists	New client library and IAM surface	**CHOSEN**
BLOBs in Postgres	One store, transactional	Bloats backups/WAL with multi-MB binaries; wrong tool for large immutable files	Rejected

> **CONFIRM** — upload a CMDB CSV, delete the API pod, re-download from a fresh pod. Team-A credentials cannot generate a working presigned URL for a team-B object.

D10 Runtime platform — Kubernetes (shape first, scheduler second)

Three Deployments (`voc-api`, `voc-worker`, `litellm-proxy`) + managed Postgres/Redis/storage. HPA scales the API; KEDA scales workers on queue depth. The three-container shape also runs on ECS or Compose — only the scheduler changes.

REASONS

- Rolling deploys, health-checked restarts, and horizontal scaling are table stakes the platform should provide, not the app.
- NetworkPolicies express the default-deny egress posture (§6) declaratively.
- Deliberately scheduler-agnostic shape: if the org is not a Kubernetes shop, nothing in this document breaks on ECS — an explicit hedge against platform lock-in.

ALTERNATIVES CONSIDERED

OPTION	PROS	CONS	VERDICT
Docker Compose on a VM	Simplest possible ops	Single node: no rolling deploys, no autoscaling, node loss = outage. Right for staging, not for "all the teams"	Staging only
Kubernetes / managed equivalent	Rolling deploys, HPA/KEDA, NetworkPolicies, org-standard tooling	Operational complexity; assumes platform competence exists in-house	**CHOSEN**
Serverless (Lambda/Cloud Run)	Zero idle cost	Long-running pipeline jobs and long-lived SSE connections are exactly the two workloads serverless handles worst	Rejected — workload mismatch

> **CONFIRM** — rolling deploy under load drops zero requests; queue backlog of 10 runs scales workers up and back down; a chaos-killed API pod recovers with no user-visible error beyond one SSE auto-reconnect.

D11 Observability — OTel traces, Prometheus metrics, Loki logs, Sentry errors

One trace spans API → queue → worker → proxy per run. Metrics: queue depth, per-stage run duration, SSE connections, token spend per team, 429s. Structured JSON logs correlated by `run_id` / `trace_id`. Sentry on both backend and frontend.

REASONS

- An LLM pipeline has two failure currencies — latency and money; both must be first-class observable from day one.
- Distributed tracing is the only tool that answers "why did the correlation agent take 4 minutes today" across a queue hop.
- The frontend currently has zero error reporting: in production, a white-screen nobody reports is an outage nobody sees.
- Spend metrics come free from the LiteLLM Proxy (D8) — the dashboards just read them.

USE CASES SERVED

Per-stage latency regressions caught on a dashboard, not by user complaint; monthly per-team cost reports; alerting on queue depth before users feel it.

> **CONFIRM** — pick any completed run id: one click from its trace shows every agent stage and every LLM call with token counts. A thrown frontend error appears in Sentry with a session-linked user id.

D12 Frontend — keep the React + Vite SPA; no Next.js

The SPA stays. Production adds: Sentry, vitest + Playwright, and login/team-context handling. The build ships to a CDN.

REASONS

- Next.js's value — SSR, SEO, first-paint for anonymous visitors — targets problems an internal authenticated tool does not have. Every page sits behind login; there is nothing to index.
- Adopting it would add a Node server tier and the RSC mental model while retiring zero ceilings from §2.
- Static output on a CDN is the cheapest, most cacheable, most secure way to ship an SPA.
- The existing investment — Cytoscape attack graph, command palette, theming, reduced-motion support — carries over untouched.

ALTERNATIVES CONSIDERED

OPTION	PROS	CONS	VERDICT
React + Vite SPA (keep)	In place, static deploy, no server tier, team fluency	No SSR (not needed)	**CHOSEN**
Next.js	SSR/RSC, file routing, image optimization	Adds a Node runtime to operate and secure; solves SEO/first-paint problems this tool does not have; migration cost with zero ceiling retired	Rejected — a solution shopping for a problem
HTMX / server-rendered	Minimal JS	The attack graph, live streams, and command palette are rich-client by nature	Rejected — fights the product

> **CONFIRM** — Playwright flow green in CI: login → launch run → watch live progress → open attack graph → export report. Lighthouse on the CDN build stays within agreed budgets.

§5 Run lifecycle: shared, synced, resumable

The product requirement driving D3, D4, and D5: when any analyst launches an analysis, every authorized teammate's UI shows the same live run — including someone who joins mid-run — and a refresh never loses the stream.

#	ACTOR → TARGET	ACTION
1	Analyst A → API	`POST /analyze` with the team's active dataset.
2	API → Redis	`SET run-lock:{team}:{dataset} NX EX ttl` — refuse with a clear message if a run is already live (the distributed `check_duplicate()`).
3	API → Postgres	Insert `runs` row: `status=queued`, requesting user, team, dataset version. Audit row written.
4	API → Redis	Enqueue the arq job; respond `202 {run_id}` to Analyst A.
5	Browser A → API	Opens `GET /runs/{id}/events` (SSE, session cookie authenticated).
6	Worker → Postgres	Dequeues; sets `status=running`; heartbeats the lock TTL.
7	Worker → PG + Redis	Per pipeline stage: append numbered event to `run_events`, then `PUBLISH run:{id}`. Durable first, live second.
8	API → Browsers	Every subscribed replica forwards events to its connected clients as SSE with the event's sequence number as `id:`.
9	Analyst B → API	Joins mid-run: API streams full history from `run_events`, then switches to the live pub/sub tail — replay-then-tail.
10	Browser A ⇢ API	On any disconnect, `EventSource` auto-reconnects with `Last-Event-ID`; API resumes from that sequence number. No gaps, no duplicates.
11	Worker → PG + Redis	Terminal state: persist result, `status=succeeded|failed|cancelled`, delete the lock, publish the terminal event.
12	Failure path	Worker death: lock TTL expires, run is marked `failed` with partial history intact; re-launch is one click. Nothing silently vanishes.

Why durable-first ordering matters (step 7): writing to Postgres before publishing guarantees a late joiner's replay is always a superset of what live viewers have seen — the invariant that makes "everyone sees the same run" true rather than approximately true.

§6 Security posture

This tool ingests the organization's CMDB and computes its attack paths. It must clear the bar it sets for everything it audits.

- **Identity everywhere.** No anonymous endpoint except the health check. SSE, chat, uploads — all behind the session (D6). Rate limits key on user id; IP-keying survives only as an unauthenticated-edge backstop.
- **Default-deny egress.** NetworkPolicies: API and workers reach Postgres, Redis, object storage, the IdP, and the LiteLLM Proxy — nothing else. Only the proxy reaches LLM providers, over an allow-list. The non-exfiltration story is provable to a security review board, not asserted.
- **Secrets.** Vault / cloud secret manager, injected at deploy. Provider keys exist only in the proxy (D8). The app's secret surface shrinks to: database URL, Redis URL, OIDC client secret, one virtual LLM key.
- **Audit trail.** Append-only `audit_log`: every mutation and every LLM-invoking action — who, what, which team, when, from where. A vulnerability platform that cannot answer "who queried what about our weakest systems" fails its own premise.
- **Data protection.** TLS everywhere external; encryption at rest on Postgres and buckets; retention policy on prompts/responses logged at the proxy — they contain infrastructure details and deserve the same classification as the CMDB itself.
- **Supply chain.** Dependency and image scanning in CI; images pinned by digest; SBOM published per release.

§7 Migration plan with phase gates

Ordered so every phase ships alone and nothing needs a big-bang cutover. A phase is done when its gate checklist is green — not before.

1 Foundations

make the codebase safe to change — no user-visible delta

WORK

- Alembic baseline migration generated from current models
- Test floor: API contract tests; orchestrator golden-run against a stubbed LLM gateway
- CI: ruff, mypy, pytest, tsc, vitest wired and required
- Dockerfiles: one image, two entrypoints (api / worker)
- Config split into environment profiles

GATE — CONFIRM BEFORE PHASE 2

- ☐ CI red blocks merge; green on main
- ☐ Golden-run test reproduces a full pipeline result offline
- ☐ `alembic upgrade head` from empty DB matches current schema
- ☐ Both containers boot from the same image in Compose

2 Externalize state — Postgres + Redis

kill Ceiling 1; still one API process, but nothing lives in it

WORK

- `DATABASE_URL` → Postgres; one-time SQLite export/import script
- `api/state.py` globals → DB rows + Redis locks
- `core/ratelimit.py` → Redis sliding window, same interface
- Sessions/coordination primitives stood up in Redis

GATE

- ☐ Two replicas behind nginx: identical state, D1's confirmation passes
- ☐ Row-count parity SQLite → Postgres per table
- ☐ PITR restore drill succeeds in staging
- ☐ Rate limits enforced identically across both replicas

3 Identity — OIDC, RBAC, audit

kill Ceiling 3 before opening the door to teams

WORK

- BFF login flow (D6); session store in Redis; auth dependency on every route
- `team_id` on datasets/runs; scoped-session dependency (D7)
- Roles mapped from IdP groups; frontend login/team context
- Append-only audit log on every mutation and LLM call

GATE

- ☐ D6 confirmation: no tokens in JS; session kill = instant 401
- ☐ D7 confirmation: cross-team access test suite fully green
- ☐ Audit row present for every mutating request in a scripted crawl
- ☐ Offboarding test: IdP-disabled user locked out within session TTL

4 Extract the pipeline — workers, events, gateway

kill Ceiling 2; the deepest change lands on solid ground

WORK

- arq worker consumes runs (D4); thread/queue bridge in `streams.py` deleted
- `run_events` + pub/sub; SSE rewritten replay-then-tail (D5)
- LiteLLM Proxy deployed; provider keys migrate out of the app (D8)
- Uploads/exports to object storage (D9)

GATE

- ☐ D4 confirmation: deploy mid-run — run and streams survive
- ☐ D5 confirmation: reconnect replay has zero gaps/duplicates
- ☐ D8 confirmation: no provider keys in app env; budget cap degrades gracefully
- ☐ Test-gap bar met: contract tests, golden-run, Playwright login → run → report

5 Production hardening

earn the pager, then invite the teams

WORK

- Kubernetes manifests; HPA + KEDA; default-deny NetworkPolicies
- Secrets from Vault; OTel + Prometheus + Loki + Sentry (D11)
- Per-team budget dashboards; alerting on queue depth and error rate
- Load test: concurrent runs + SSE fan-out at 5× expected peak

GATE

- ☐ D10 confirmation: zero-drop rolling deploy; chaos-kill recovery clean
- ☐ Backup restore drill repeated on production infrastructure
- ☐ Staged rollout: two pilot teams for two weeks before general availability
- ☐ Runbook written: on-call can restore service from each single-service outage

§8 Risk register

RISK	SEVERITY	MITIGATION
Tenancy scoping bug leaks one team's findings to another	HIGH	Single enforcement point (D7); cross-team access test suite required green in CI from Phase 3 onward; audit log makes any incident investigable.
LLM spend runs away under multi-team load	HIGH	Hard per-team budgets at the proxy (D8) with graceful degradation; spend dashboards and alerts (D11) before general availability.
Test debt makes Phase 4's deep refactor regress silently	HIGH	Phase 1 builds the floor; Phase 4's gate explicitly requires the golden-run and Playwright suites. The refactor does not ship without them.
Redis outage during live runs	MED	Nothing durable in Redis by construction (D3); runs fail visibly and are re-launchable; managed Redis with failover in production.
IdP integration friction (claims, groups, redirect URIs)	MED	Phase 3 starts against a local Keycloak with the same claim shapes; corporate IdP wired once flows are proven.
Org platform mismatch (no Kubernetes competence)	LOW	Scheduler-agnostic three-container shape (D10); ECS/Compose fallback documented.
Migration fatigue — phases stall after Phase 2	LOW	Each phase ships standalone value (Phase 2 alone enables replicas; Phase 3 alone enables teams); no phase depends on a future one to be useful.

§9 Appendix

A. Application secret surface (after Phase 4)

SECRET	HELD BY	ROTATION
`DATABASE_URL`	api, worker	Secret manager; rolling restart
`REDIS_URL`	api, worker	Secret manager; rolling restart
OIDC client secret	api	IdP + secret manager, coordinated
LLM virtual key (per env)	api, worker	Proxy-side; zero app deploys
Object-storage credentials	api, worker	IAM-scoped per team prefix
Provider keys (Anthropic, Mistral, …)	**LiteLLM Proxy only**	Proxy secret mount; invisible to the app

B. Capacity assumptions (initial sizing, revisit after pilot)

- Teams: 5–15 at general availability; concurrent live runs: ≤ 1 per team by design (dataset lock), so ≤ 15 system-wide.
- Run duration: minutes-scale, LLM-latency-bound — worker count M sized to team count, not CPU.
- SSE fan-out: tens of concurrent viewers per run worst case; hundreds of connections system-wide — trivial for N=2–3 API replicas.
- Postgres: `run_events` is the only fast-growing table; partition or archive beyond 90 days.
- Redis: coordination-only working set — smallest managed tier suffices; failover enabled in production.

C. What deliberately stays (unchanged from the current build)

- **FastAPI** (D1), **React + Vite SPA** (D12), **litellm client** (D8 makes it point at the proxy).
- **The seven-agent pipeline** — orchestrator logic changes process, not shape; grounding, chaining, and the held-out-oracle verification design are untouched.
- **networkx + Cytoscape** — graph reasoning and rendering are orthogonal to deployment topology.
- **SQLAlchemy models** — ported to Postgres via connection string; Alembic formalizes what exists.

The whole document in one sentence: keep the stack, evict the state — Postgres for truth, Redis for now, workers for work, OIDC for who, a gateway for spend — in five gated phases that each ship alone.

`VOC-ARCH-001 v1.0 · 17 July 2026 · generated from the living architecture document; supersedes the HTML draft of the same date.`